

Automated Component Recovery and Custom Kit Assembly System for Educational Circuit Kits

1st Aman Mishra
Computer Eng. and Informatics
Middlesex University Dubai
Dubai, UAE
a.mishra@mdx.ac.ae

2nd Faseeh Mohammed
Computer Eng. and Informatics
Middlesex University Dubai
Dubai, UAE
ff416@live.mdx.ac.uk

3rd Judhi Prasetyo
Computer Eng. and Informatics
Middlesex University Dubai
Dubai, UAE
j.prasetyo@mdx.ac.ae

Abstract—Educational institutions issue circuit development kits, such as Arduino modules, for practical coursework, but returned kits often lack critical components and cannot be redistributed as complete sets. This paper proposes an automated robotic solution to this logistical and environmental inefficiency. Unlike traditional assembly systems that build standard products from uniform supply lines, the proposed system recovers components from existing, incomplete kits. A Python machine-vision system identifies available modules and communicates via TCP/IP with an EPSON RC8.0 simulator controlling a VT6-A901S 6-axis robot, enabling dynamic assembly of new kits from custom Bills of Quantities (BOQs) submitted for specific projects. The result is a flexible, waste-reducing system that gives students exactly the components their modules require.

Index Terms—Robotic Assembly, Machine Vision, Component Recovery, EPSON RC8.0, Automation, E-waste.

I. INTRODUCTION

Practical electronics and robotics education relies heavily on development kits, typically built around microcontrollers such as the Arduino alongside sensors, actuators, and passive components. Although students are encouraged to keep these kits, many return them at the end of term, often with modules missing, damaged, or misplaced. Because the returned kits are incomplete, they cannot be re-issued to incoming students who need standard, comprehensive sets, so institutions accumulate partial kits, incurring unnecessary procurement costs and electronic waste.

This project develops a robotic system that sorts returned components and dynamically rebuilds kits. By targeting project-specific requirements rather than rigid standard kit definitions, the system reallocates whatever components a returned kit contains to the students whose projects actually need them, even when generic parts are absent.

II. METHODOLOGY

A. Evaluation of Existing Solutions

Automated kitting and assembly are well established but almost exclusively designed for factory-stage production, typically using Delta and SCARA (Selective Compliance Assembly Robot Arm) robots to pick uniform components from controlled feeder lines. Automated sortation is also used in large-scale waste management: facility-scale smart sortation systems combine real-time machine vision with air jets and

Delta arms to recover valuable commodities, including e-waste, from mixed municipal solid waste [1]. These systems prove the efficacy of AI-guided robotic sorting for high-volume recovery, but operate at facility scale for gross sorting rather than the precise, project-specific kitting of delicate microelectronic modules needed in education.

These methodologies share one assumption: a continuous, uniform supply of new components. No widely adopted solution recovers loose, mixed components from partially depleted kits to build newly configured ones. Recent work on e-waste recovery stresses that salvaging used electronics differs fundamentally from original production: recovery is characterized by high uncertainty, as components may be damaged, randomly oriented, or missing [2]. Deterministic factory automation is therefore insufficient for these Re-X (Recycling, Re-use, Remanufacturing) activities, motivating the flexible, data-driven approach proposed here.

B. Proposed Unique Approach

The solution's differentiating factor is intelligent recovery and flexible kitting. Rather than having staff manually audit and sort thousands of components to rebuild standard kits, the system operates on a custom Bill of Quantities (BOQ) model: staff or students submit a project BOQ, and the system identifies the required parts from the pool of returned components and assembles a kit tailored to that project. This shifts the institution from a “push” model (a standard kit for everyone regardless of need) to a “pull” model (kits assembled strictly from project requirements), maximizing the utility of returned inventory.

III. IMPLEMENTATION

To realize this automated recovery and assembly system, a simulated robotic workcell has been developed, integrating high-level software control with industrial robotic simulation.

A. Machine Vision System

The machine-vision architecture is the core of the pick-and-place operation: it scans the unstructured workbench, identifies components, decodes their orientation, and translates pixel data into Cartesian coordinates for the EPSON VT6-A901S manipulator. Many industrial bin-picking solutions rely on

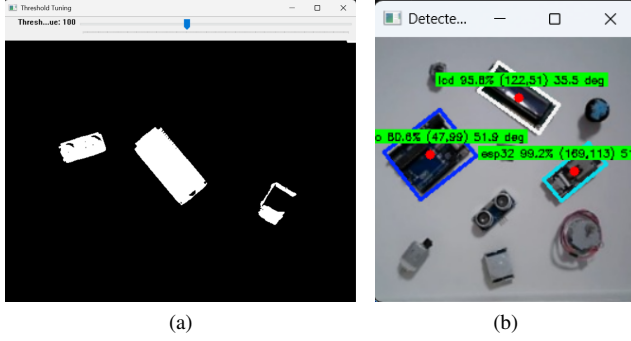


Fig. 1. Evolution of the perception stage, shown on comparable workbench scenes. (a) Prototype: global binary thresholding in OpenCV, tuned live by an operator slider. The mask isolates the three target modules but fragments them where glare erases surface texture, and carries no class or orientation information. (b) Deployed detector: YOLOv8-OBB inference on the raw frame, recovering class, confidence, centroid and resting angle for each module in a single forward pass while ignoring the surrounding clutter.

line-of-sight 3D cameras, but these generate point clouds prone to spurious outliers on cluttered, overlapping parts, which degrade obstacle-avoidance planning and segmentation and require costly filtering [3]. To avoid 3D point-cloud filtration and keep real-time processing efficient, this project uses a 2D planar overhead approach. The system developed through two phases, culminating in a deep-learning solution.

1) *Prototype Phase: Deterministic Computer Vision*: The initial architecture relied on deterministic OpenCV processing. Frames from an overhead 720p webcam were Gaussian-blurred to suppress noise, then global binary thresholding (Fig. 1a) isolated geometric footprints. Contour analysis (`cv2.findContours`) found each object’s boundary, and a scale-preserving script padded the crops into uniform 224×224 tensors for a Google Teachable Machine classifier. Although functional in controlled conditions, the prototype proved fragile: metallic glare erased surface texture, while background clutter and cast shadows on the light bench produced spurious contours. A rotated rectangle fitted to each contour did recover a resting angle, but that angle was only ever as reliable as the binary mask it was fitted to. The pipeline also needed a separate five-class classifier to label each crop, and was superseded before the contour-to-classifier link was completed. Section IV-C quantifies these limits against the deployed detector.

2) *Final Architecture: YOLOv8-OBB (Oriented Bounding Box)*: To overcome these limitations, the system was re-architected around a YOLOv8-OBB neural network that ingests the full-resolution, uncropped camera frame, eliminating manual thresholding and cropping. Trained on a purpose-built dataset annotated with the Computer Vision Annotation Tool (CVAT) under photometric and geometric augmentation (Section III-A3), the OBB model bypasses glare and localized clutter (Fig. 1b). The manual annotation this supervised approach requires is a known but necessary trade-off for the recognition quality and spatial accuracy that bin-picking demands [4]. In a single forward pass it outputs a 5-D oriented tensor ($cx, cy, width, height, theta$) with multi-class

TABLE I
DATASET AND TRAINING CONFIGURATION

Parameter	Value
<i>Dataset</i>	
Images (train / validation)	50 / 6
Instances (train)	132 (44 per class)
Classes	Arduino, ESP32, LCD
Annotation	CVAT, oriented bounding box
Capture	Overhead 720p webcam, deployment cell
<i>Model and optimisation</i>	
Network	YOLOv8n-OBB, COCO-pretrained
Input resolution	640×640
Epochs / batch size	100 / 16
Optimiser	AdamW (auto-selected)
Learning rate (initial / final)	1.43×10^{-3} / 1.43×10^{-5}
Schedule	Linear decay, 3 warm-up epochs
Momentum / weight decay	$0.9 / 5 \times 10^{-4}$
Loss weights (box/cls/dfl/angle)	7.5 / 0.5 / 1.5 / 1.0
Training time	151 s (Colab T4 GPU)
<i>Train-time augmentation</i>	
HSV hue / saturation / value	0.015 / 0.7 / 0.4
Translation / scale	0.1 / 0.5
Horizontal flip / mosaic	0.5 / 1.0 (off for last 10)
Random erasing	0.4
Rotation / shear / perspective	disabled

probabilities (Arduino, ESP32, LCD). This is far simpler than hybrid pipelines that, for example, use YOLOv5 for top-view detection followed by an eye-in-hand FAST/BRISK feature-matching step for pose estimation [5]. By deriving the resting angle (θ) from a single overhead frame, the OBB architecture removes any dependence on perfect lighting or multi-step image capture.

3) *Dataset and Training Configuration*: Table I lists the full configuration. The detector was trained on a purpose-built dataset of 56 overhead images captured with the deployment camera on the workcell itself, so that the training and operating distributions coincide by construction. Components were placed at varied positions and resting yaw angles across captures, and oriented bounding boxes for the three classes were annotated in CVAT. The set was split 50/6 into training and validation, yielding 132 training instances balanced at 44 per class.

Training began from COCO-pretrained `yolov8n-obb` weights and ran for 100 epochs at 640 px. Automatic optimiser selection resolved to AdamW at an initial learning rate of 1.43×10^{-3} , decayed linearly to one percent of that value over three warm-up epochs and 97 training epochs. The default photometric and geometric augmentation pipeline was applied—HSV jitter, mosaic composition (disabled for the final ten epochs), random translation and scaling, horizontal flipping, and random erasing. Rotational augmentation was deliberately left disabled: orientation diversity is supplied by the physical placement of components during capture, which samples the true distribution of resting angles more faithfully than synthetic rotation of a fixed overhead view. The full run completed in 151 s on a single Colab GPU.

4) *Logistics Filtering and BOQ Engine*: Once the vision system extracts each component’s pixel centroid (X, Y) and class label, a logistics filter operates above the frame

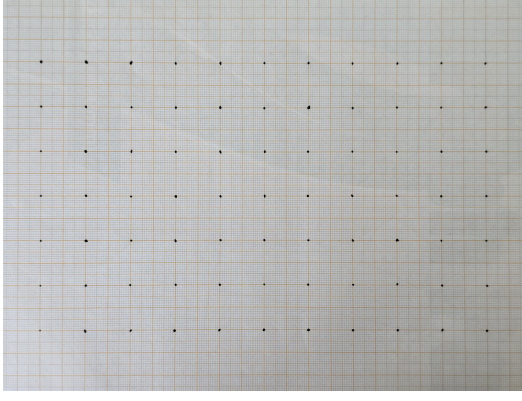


Fig. 2. 77-point homography calibration grid showing pixel click positions overlaid on the workspace reference image.

buffer. Rather than greedily forwarding every detection to the coordinate-mapping pipeline, the orchestrator matches the detected inventory against a Bill of Quantities (BOQ), ignoring surplus components even when they are validly detected, so that only the required, correctly kitted items are passed to the spatial calibration engine for EPSON translation.

B. Spatial Calibration and Coordinate Mapping

A critical requirement is translating pixel positions in the camera image into physical millimetre coordinates in the robot's world frame, achieved through a planar homography that provides a projective mapping between the two coordinate spaces.

The mapping uses a 77-point calibration grid of 11 columns by 7 rows at 20 mm spacing in both axes (Fig. 2). The world X-coordinate decreases from +100 mm to -100 mm left to right and the world Y-coordinate increases from 410 mm to 530 mm top to bottom, giving a calibrated area of 200 mm $\times$ 120 mm. A reference image of the grid (`graph_paper.jpg`) is captured with the camera normal to the workspace; `tools/calibration_clicker.py` records the 77 intersections, and `tools/sort_and_tag_pixels.py` sorts them column by column and appends the world coordinates to produce a four-column pixel-to-world CSV.

The homography matrix $\mathbf{H}$ is computed with OpenCV's `findHomography` using standard least squares (SLS), not RANSAC. This is deliberate: RANSAC rejects outliers, but the corner points define the workspace extremes and are valid, important constraints, so treating them as outliers would introduce systematic error near the boundaries. With 77 evenly distributed points, SLS gives a robust estimate without outlier rejection. At runtime, any detected centroid (p_x, p_y) is mapped to world coordinates (w_x, w_y) via `cv2.perspectiveTransform` with $\mathbf{H}$.

Repositioning the camera vertically changes the image scale and invalidates the pixel-to-millimetre mapping. Rather than re-clicking all 77 intersections, a recalibration routine lets the user mark two points a known 20 mm apart and derives a scale factor $s = d_{\text{measured}}/d_{\text{expected}}$, where the expected pixel distance $d_{\text{expected}} = \sqrt{(r_h \cos \theta)^2 + (r_v \sin \theta)^2}$ uses the separate

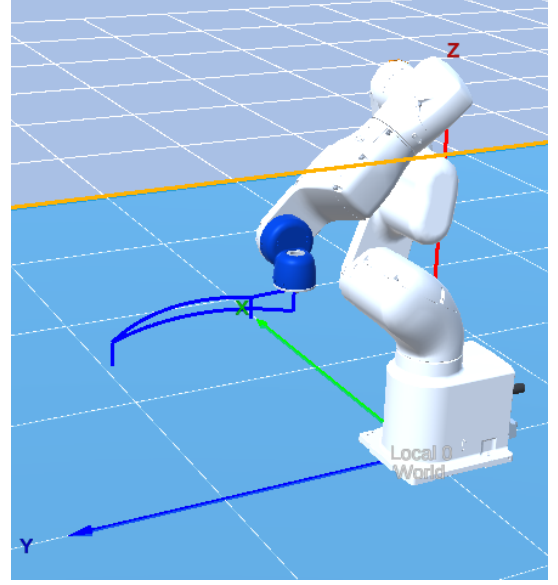


Fig. 3. Trajectory of the PICK command: three-phase Jump3 approach to the component, gripper activation, transport to PlacePoint, and release.

horizontal and vertical pixel pitches r_h, r_v of the original grid. This anisotropic form accommodates lens geometry and non-square sampling. All 77 coordinates are then scaled about the grid centre by s and cached for subsequent runs.

C. TCP/IP Communication Protocol

Fig. 3 shows a sample pick-and-place for one item: from standby, the arm approaches the object in the correct orientation, picks it, jumps to the place point, fixes orientation, places, and retracts to standby for the next operation.

The Python control system communicates with the EPSON RC8 controller over a local TCP socket (127.0.0.1:2001 in simulation, a LAN address for the physical robot). Commands are plain ASCII strings of the form `COMMAND X Y Z U\r\n`, where X, Y, Z are world coordinates in millimetres and U is the tool rotation in degrees. The supported set is JUMP, GO, MOVE, PICK and STANDBY; the controller replies "OK" after each, and a small inter-command delay allows mechanical completion before the next is sent.

For multi-component picks, `epsonPickAll` iterates over a list of world-coordinate strings and issues a PICK for each, checking the reply against "OK". Any non-OK response aborts the batch immediately and returns the robot to standby, so a partial failure such as a grip error or controller fault cannot leave the robot operating in an undefined state.

D. EPSON RC8 Robot Program

`Epson/Pickerbot_Receiver/Main.prg`), the robot-side program is written in Epson SPEL+, the RC8 controller's scripting language, building on a template script [6] with predefined motion-initialization parameters. After initialization it opens a TCP server on channel #201 at port 2001 and waits for a client before entering its command loop.

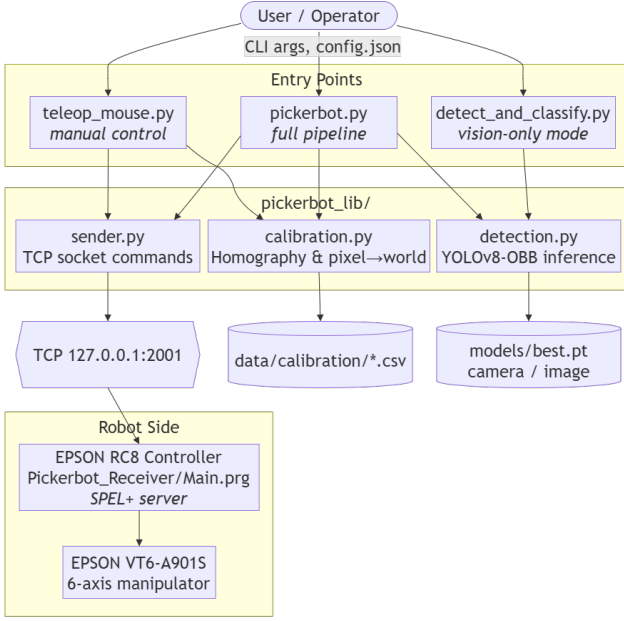


Fig. 4. Software architecture of the Picker-Bot system, illustrating the three entry-point scripts and their shared dependency on the `pickerbot_lib` package.

Each string is split on spaces into the command token and floating-point coordinate arguments. The `PICK` command is the most complex: a three-phase Jump3 trajectory lifts the arm vertically, translates to an elevated waypoint above (x, y) with the tool at orientation u , and descends to the pick position, where a digital output closes the gripper and a dwell ensures a reliable grip. A second Jump3 lifts clear and transports the component to a predefined `PlacePoint` (the target kit container), where the output is deactivated to release it before the arm returns to standby.

A network error handler reopens the server socket and resumes listening rather than halting, letting the system recover from transient network interruptions without operator intervention.

E. Software Architecture and Entry Points

The codebase (Fig. 4) is built around a shared package, `pickerbot_lib`, consolidating detection, calibration, and TCP communication into three submodules: `detection`, `calibration`, and `sender`. Three entry-point scripts at the project root each import this package to serve a distinct mode.

The primary entry point, `pickerbot.py`, orchestrates an end-to-end pick session. A `--src` argument selects the detection source (live camera or a static image), and `--calibrate` launches the height-recalibration GUI and exits. In a normal run it loads the homography CSV named in `config.json`, runs detection to obtain centroid pixels, maps each to world coordinates, and dispatches the list to `epsonPickAll` over TCP.

The second, `detect_and_classify.py`, isolates the vision layer for development and testing without a robot

TABLE II
QUANTITATIVE EVALUATION

(a) Detection — validation split, 6 images / 18 instances					
Class	Inst.	P	R	mAP_{50}	mAP_{50-95}
Arduino	6	0.977	1.000	0.995	0.995
ESP32	6	1.000	1.000	0.995	0.955
LCD	6	0.975	1.000	0.995	0.971
All	18	0.984	1.000	0.995	0.974
(b) Vision throughput — per frame					
Pre-process / inference / post-process				0.3 / 13.9 / 2.7 ms	
Total				16.9 ms (59 fps)	

connection: given a file path it processes a static image, and without arguments it opens the live feed with a real-time confidence-threshold slider.

The third, `teleop_mouse.py`, provides interactive positioning: it loads the calibration, connects to the robot, and displays the reference image with all 77 points annotated. Each click is transformed through H and sent as a `MOVE` command, enabling manual positioning without the full pipeline, which is useful for verifying calibration accuracy before an automated session.

F. Robotic Control and Simulation

Manipulation is simulated in the EPSON RC8.0 environment. The system can cover any workspace size; the initial setup mapped a 200 mm $\times$ 120 mm evaluation space with a deposit point 280 mm above its centre. The platform is the EPSON VT6-A901S [7], a 6-axis arm with 980 mm reach suited to pick-and-place requiring varied approach angles and high repeatability; full installation, safety, and operational specifications appear in the Epson VT6 documentation [8].

IV. RESULTS

The Picker-Bot system integrated deep-learning vision with industrial robotic control inside the EPSON RC8.0 simulator. The following subsections report observed behaviour across three layers: perception, spatial reasoning, and supervised manipulation.

A. Detection Performance

The migration from the OpenCV prototype to the YOLOv8-OBb architecture substantially improved detection reliability. The final model classified Arduino, ESP32, and LCD modules and recovered the resting angle θ with the centroid (cx, cy) in a single forward pass, with no geometric post-processing, and absorbed the glare, clutter, and background texture that had destabilised the prototype. The explicit “noise” class that the prototype classifier had required was no longer necessary: the detector is trained on the three component classes alone and treats everything else as background, so spurious regions are rejected by the confidence threshold rather than by a dedicated reject class.

Table II(a) reports per-class validation metrics. The model reached an mAP_{50} of 0.995 at 1.000 recall and 0.984 precision. The normalised confusion matrix is fully diagonal: no

TABLE III
DETERMINISTIC BASELINE VS. YOLOv8-OBb, FIVE WORKBENCH SCENES

	Threshold + contour	YOLOv8-OBb
Scenes correct, per-scene tuning	5 / 5	—
Scenes correct, one global setting	4 / 5	5 / 5
Stable parameter window	narrows with clutter	conf. 0.20–0.44
Spurious regions at best setting	3	0
Class label	separate classifier	joint
Orientation θ	fitted to mask	regressed

inter-class confusion, and no component predicted as background. The mAP_{50-95} of 0.974 is more significant because OBb matching uses *rotated* IoU—a detection whose yaw is substantially wrong fails to match its ground truth at the higher IoU thresholds and depresses this metric. A high value therefore certifies that the recovered angle θ , and not merely the class label, is accurate. Convergence was rapid: mAP_{50} exceeded 0.99 by epoch 7 and mAP_{50-95} passed 0.90 by epoch 17, the remaining epochs refining localisation. A fixed confidence threshold of 0.65 eliminated the few remaining glare-induced false positives, with no additional non-maximum suppression required.

Perception is not the throughput bottleneck. End-to-end vision cost is 16.9 ms per frame (Table II(b)), around 59 fps; cycle time is dominated by the Jump3 trajectory and the fixed inter-command delay discussed in Section VI-A.

Scope of this evaluation: The validation split contains 6 images and 18 instances. No separate test partition was withheld since the dataset was captured in the same cell the system operates in, so the same images that measure performance also informed checkpoint selection. These figures are therefore an in-distribution characterisation of the deployment cell, not an estimate of generalisation to new environments. At this sample size a recall of 18/18 carries a 95% Wilson confidence interval of [0.82, 1.00], and precision cannot resolve differences of a few percent. In-distribution accuracy is the operationally relevant quantity; the interval, rather than the point estimate, is the honest summary of it.

B. Spatial Accuracy of the Pixel-to-World Mapping

The homography reliably bridged the camera image and the planar workspace, with the 77-point grid defining a calibrated region of roughly 200 mm $\times$ 120 mm that covered the full overhead field of view. Using SLS rather than RANSAC kept the boundary corner points from being discarded as outliers, which would otherwise have contracted the calibrated area.

Height recalibration absorbed camera repositioning in seconds without re-clicking every intersection: measuring a known 20 mm segment against the original grid pitches recovered an anisotropic scale factor applied about the grid centre. The result was verified through `teleop_mouse.py`, where each click produced a MOVE command whose terminal position visually matched the clicked feature.

C. Comparison with the Deterministic Baseline

To quantify why the deterministic pipeline was abandoned, it was re-run alongside the deployed detector on five held-

out workbench scenes containing 1–11 components each. The contour stage’s area bounds were scaled proportionally to image area from the values used at 720p, and the binary threshold was swept across its full range, so the baseline is credited with its best achievable result rather than one arbitrary setting.

Given per-scene tuning, the baseline recovers the correct number of regions in all five scenes but the algorithm is unstable with no single threshold solution. The best global value succeeds on four scenes and over-segments the fifth into 14 regions against 11 true components, the surplus being glares and shadows. The usable threshold window also collapses as scene gets cluttered—130 of 255 values for a single component, 74 for four, and 15 and 8 for the two eleven-component scenes—so constant manual retuning between scenes is required, deviating from the original intention of the system.

The learned detector behaves in the opposite way. Any fixed confidence between 0.20 and 0.44 recovers all 13 target-class instances across all five scenes with no false positives, and the deployed value of 0.65 recovers 12 of 13; the single miss is an ESP32 returned at 0.444 confidence rather than a localisation failure. The difference defines the parameter stability and not peak accuracy, i.e., one confidence value transfers unchanged across every scene. The baseline being class-agnostic, proposed regions without labels to a separate classifier for identification, and thus, was never implemented.

D. Integrated Robot Control and Logistics

The control pipeline executed the pull-based kitting end-to-end. Given a BOQ, the system satisfied its requirements in order of detection confidence and ignored correctly recognised but unneeded components, behaving as a logistics auditor rather than a greedy collector; without a BOQ it reverted to greedily picking every detection above threshold.

The TCP/IP interface dispatched the full command set (JUMP, GO, MOVE, PICK, STANDBY) reliably across extended runs, each validated by an OK reply before the next was sent. The PICK primitive executed its three-phase Jump3 trajectory in every trial, and synthetic failures exercised the abort path, returning the manipulator to standby rather than continuing through an undefined state. These are functional verifications of the control layer in simulation rather than a measured grasp-success rate, which requires the physical cell (Section VI-A). Across all configurations the system switched cleanly between live USB and pre-recorded images via a single `config.json` setting, interfaced with the RC8.0 simulator at 127.0.0.1, and kept 0.65 as the default confidence threshold while exposing a runtime slider.

V. FUTURE WORK

The most immediate step is deployment to physical hardware: moving from the RC8.0 simulator to a real VT6-A901S requires calibrating the physical camera under varying lighting, tuning gripper timing to real component dynamics, and likely an adaptive or compliant gripper for delicate modules. That deployment is also what enables the two measurements

this study could not make—a held-out evaluation across capture sessions, and a grasp-success rate. Further ahead, a web portal would let students submit BOQs directly and collect completed kits, while Internet of Things technologies and Autonomous Guided Vehicles, increasingly used to route materials in flexible manufacturing [9], could fetch bins of returned kits to the workcell and deliver finished kits to collection points.

VI. CONCLUSION

The Picker-Bot system successfully shifted component kitting from a “push” model to a project-specific “pull” model. Its technical cornerstone was the migration from deterministic OpenCV prototypes to YOLOv8-OBb: where the prototype was highly sensitive to environmental shifts, the final model reached an mAP_{50} of 0.995 and an mAP_{50-95} of 0.974, while the 77-point planar homography mapped the full overhead field of view onto the workspace and was verified interactively through click-to-move positioning, showing that oriented bin-picking is viable in cluttered educational settings without 3D point-cloud hardware. The “logistics auditor” approach—filtering detections through the BOQ engine—lets the system both detect parts and manage inventory, directly addressing the goal of reducing procurement costs and e-waste.

A. Limitations

Two limits bound what the reported numbers support. The detector was evaluated on a 6-image, 18-instance validation split drawn from the same capture session as the training data and not withheld from checkpoint selection, so it characterises in-distribution accuracy in the deployment cell rather than generalisation; a held-out session, and ideally a second lighting configuration, is the next measurement to take. Separately, the manipulation layer is verified functionally in simulation but its grasp-success rate is not quantified, since a meaningful figure requires the physical robot and real components.

Several further constraints would affect any physical deployment. The fixed software delay after each OK kept the simulated pipeline stable but should instead be tuned to each component’s inertia, since a heavier LCD settles differently from a lighter ESP32 and a single value trades throughput against reliability. The 77-point homography is a planar model of a three-dimensional problem: components of non-negligible height introduce a parallax offset that appears as a small systematic lateral error during the Jump3 descent, so a height-aware correction (per-class offsets or a depth sensor) would be needed. Finally, the abort-on-failure strategy—correct as an industrial default—is blunt at higher throughputs, where a graduated model distinguishing transient communication faults from genuine mechanical errors would be necessary to scale to production volumes.

B. Concluding Remarks

The Picker-Bot system provides a technically sound, sustainable solution for managing returned educational circuit kits. By integrating Oriented Bounding Box detection, planar

homography, and TCP/IP control, it shows that modern AI can turn “loose waste” into project-ready assets, setting a precedent for “Re-X” (Recovery and Re-use) activities in microelectronic environments and bridging waste management with logistical efficiency.

ACKNOWLEDGMENT

The authors acknowledge the guidance and resources of the PDE4435 module, which encouraged a design-thinking approach to advanced robotics projects, and the use of Claude code to set up the baseline masking filters for detection with sliders for human-operated tuning before transitioning to YOLOv8-OBb.

REFERENCES

- [1] AMP, “AMP ONE: Automated Waste Sorting Systems & Facility,” 2026. [Online]. Available: <https://ampsortation.com/products/amp-one>. [Accessed: Apr. 17, 2026].
- [2] J. Saenz *et al.*, “Automated disassembly of e-waste—requirements on modeling of processes and product states,” *Frontiers in Robotics and AI*, vol. 11, p. 1303279, 2024.
- [3] M. Franaszek, P. Rachakonda, and K. S. Saidi, “Filtering Organized 3D Point Clouds for Bin Picking Applications,” *Applied Sciences*, vol. 14, no. 3, p. 961, Jan. 2024.
- [4] P. Fager, R. Hanson, Å. Fasth-Berglund, and S. Ekered, “Supervised and unsupervised learning in vision-guided robotic bin picking applications for mixed-model assembly,” *Procedia CIRP*, vol. 104, pp. 1304–1309, 2021.
- [5] T. Taweesoontorn, S. Yanyong, and P. Konghuayrob, “Hybrid Deep Learning and FAST-BRISK 3D Object Detection Technique for Bin-picking Application,” *Sensors and Materials*, vol. 36, no. 4, pp. 1389–1404, 2024.
- [6] J. Prasetyo, “RC7_Python,” Jan. 03, 2024. Available: https://github.com/judhi/RC7_Python. [Accessed: Apr. 07, 2026].
- [7] EPSON Robotics, “RC8.0 Reference Manual,” EPSON Factory Automation, 2023. [Online]. Available: <https://www.epson.com>
- [8] Epson America, Inc., “Epson VT6L All-in-One 6-Axis Robots — Support,” Epson US, 2026. [Online]. Available: https://epson.com/Support/Robots/6-Axis-Robots/VT-Series/Epson-VT6L-All-in-One-6-Axis-Robots/s/SPT_VT6L-A901SS. [Accessed: Apr. 07, 2026].
- [9] I. Vlachos, R. M. Pascuzzi, M. Ntosis, K. Spanaki, S. Despoudi, and P. Repoussis, “Smart and flexible manufacturing systems using Autonomous Guided Vehicles (AGVs) and the Internet of Things (IoT),” *International Journal of Production Research*, pp. 1–22, 2022.